

RxJS Operators Cheat Sheet for E-commerce app

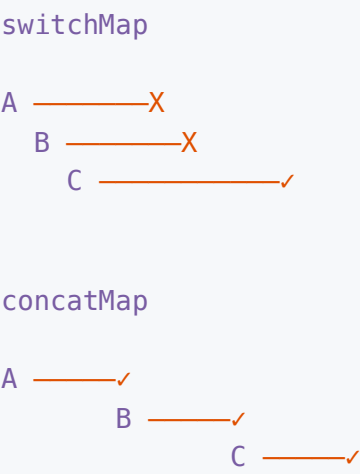
1. The 4 Higher-Order Mapping Operators

Operator	Mental Model	E-commerce Use Case	What happens to previous work?
switchMap	 Switch	Product search, filters	Cancels/unsubscribes previous inner observable
concatMap	 Queue	Ordered cart updates, checkout	Waits for previous to complete
mergeMap	 Parallel	Independent operations	Runs concurrently
exhaustMap	 Busy	Payment, Add to Cart	Ignores new emissions while busy

Memory Trick

switchMap → SWITCH → cancel old
concatMap → QUEUE → wait
mergeMap → MERGE → parallel
exhaustMap → BUSY → ignore new

Visual



mergeMap

A ————— ✓
B ————— ✓
C ————— ✓

exhaustMap

A ————— ✓
B — ignored
C — ignored

2. Product Search

Scenario

User types:

lap → lapt → lapto → laptop

Operators

```
searchControl.valueChanges.pipe(  
    debounceTime(300),  
    distinctUntilChanged(),  
    filter(query => query.length >= 2),  
    switchMap(query =>  
        productService.search(query)  
    )  
);
```

Flow

```
User typing  
  ↓  
debounceTime  
  ↓  
distinctUntilChanged
```

```
↓  
filter  
↓  
switchMap  
↓  
API
```

Remember

Search = **debounce** + **distinct** + **switch**

3. Product Search + Multiple Filters

Filters:

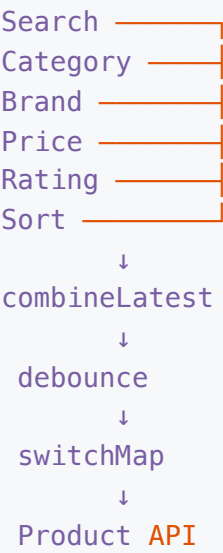
```
Search  
Category  
Brand  
Price  
Rating  
Sort
```

Use:

```
combineLatest([  
  search$,  
  category$,  
  brand$,  
  price$,  
  rating$,  
  sort$  
]).pipe(  
  debounceTime(300),  
  switchMap(([search, category, brand, price, rating, sort]) =>  
    productService.search({  
      search,  
      category,  
      brand,  
      price,  
      rating,  
      sort  
    })  
  )
```

```
)  
);
```

Flow



Remember

Multiple live inputs = `combineLatest`

4. `combineLatest` vs `forkJoin`

	<code>combineLatest</code>	<code>forkJoin</code>
Purpose	Reactive combination	Wait for all to finish
Emits	Whenever any source changes	Once
Continuous streams	✓	✗
HTTP APIs	Sometimes	Excellent
Product filters	✓	✗
Homepage APIs	Usually ✗	✓

Memory

```
combineLatest → LIVE  
forkJoin      → DONE
```

5. Product Details

Independent APIs:

```
Product  
Reviews  
Recommendations
```

Use:

```
forkJoin({  
  product: productService.getProduct(id),  
  reviews: reviewService.getReviews(id),  
  recommendations: recommendationService.getRecommendations(id)  
});
```

Memory

`forkJoin` = wait for everybody

6. Dependent APIs

```
Product  
  ↓  
Seller ID  
  ↓  
Seller  
  ↓  
Seller Reviews
```

Use:

```
product$.pipe(
  switchMap(product =>
    sellerService.getSeller(product.sellerId)
  ),
  switchMap(seller =>
    reviewService.getReviews(seller.id)
  )
);
```

Memory

API B needs API A = `switchMap`

7. Cart Quantity Updates

User clicks:

+ + + +

If every update must be processed in order:

```
quantityChange$.pipe(
  concatMap(change =>
    cartService.updateQuantity(change)
  )
);
```

Flow

```
+1 ———✓
      +1 ———✓
            +1 ———✓
                  +1 ———✓
```

Memory

Order matters = `concatMap`

8. Add to Cart

Prevent duplicate clicks:

```
addToCartClick$.pipe(  
  exhaustMap(product =>  
    cartService.add(product)  
  )  
);
```

Flow

```
CLICK ————— API ————— ✓  
CLICK ————— ignored  
CLICK ————— ignored
```

Memory

Button must not execute again while busy = `exhaustMap`

9. Payment

```
payClick$.pipe(  
  exhaustMap(() =>  
    paymentService.pay()  
  )  
);
```

Why?

Prevent:

```
PAY  
PAY  
PAY
```

from creating multiple payment requests.

Memory

Payment = BUSY → IGNORE

10. Checkout

Typical:

```
Validate Cart
  ↓
Reserve Inventory
  ↓
Calculate Tax
  ↓
Create Order
  ↓
Payment
```

Use sequential processing where operations are dependent/order-sensitive:

```
validateCart$.pipe(
  concatMap(() => reserveInventory()),
  concatMap(() => calculateTax()),
  concatMap(() => createOrder()),
  concatMap(() => processPayment())
);
```

Memory

Checkout pipeline = **ordered operations**

11. Infinite Scroll

```
Scroll
  ↓
Near bottom?
  ↓
API
```

↓
Append results

```
loadMore$.pipe(  
  exhaustMap(page =>  
    productService.getProducts(page)  
  ),  
  scan(  
    (all, response) => [...all, ...response.products],  
    []  
  )  
);
```

Operators

```
filter      → Are we near bottom?  
exhaustMap → Prevent duplicate requests  
scan       → Accumulate products
```

12. Order Tracking

Poll every 10 seconds:

```
timer(0, 10000).pipe(  
  switchMap(() =>  
    orderService.getStatus(orderId)  
  ),  
  distinctUntilChanged(  
    (a, b) => a.status === b.status  
  ),  
  takeUntil(destroy$)  
);
```

Flow

```
timer  
↓  
API  
↓
```

```
status changed?
```

↓

```
UI
```

Memory

```
Polling = timer + switchMap
```

13. Live Inventory

WebSocket:

```
inventoryUpdates$.pipe(  
  filter(update => update.productId === productId),  
  map(update => update.quantity),  
  distinctUntilChanged()  
);
```

Operators

```
filter → relevant product  
map → extract quantity  
distinctUntilChanged → ignore same quantity
```

14. API Cache

```
categories$ = categoryService.getCategories().pipe(  
  shareReplay({  
    bufferSize: 1,  
    refCount: true  
  })  
);
```

Flow

```
API — shareReplay — Component A
                        |
                        | Component B
                        |
                        | Component C
```

Memory

`shareReplay` = share + remember

15. Action + Current State

Example:

```
Add To Cart
+
Selected Product
```

Use:

```
addToCartClick$.pipe(
  withLatestFrom(selectedProduct$),
  switchMap(([, product]) =>
    cartService.add(product)
  )
);
```

Memory

Action happens → take latest state = `withLatestFrom`

16. Reactive Price Calculation

```
combineLatest([
  subtotal$,
  discount$,
  tax$,
  shipping$
]).pipe(
```

```
map([subtotal, discount, tax, shipping]) =>
  subtotal - discount + tax + shipping
);
```

Memory

Multiple changing values → calculation = `combineLatest + map`

17. Coupon Validation

```
couponControl.valueChanges.pipe(
  debounceTime(500),
  distinctUntilChanged(),
  filter(code => code.length >= 3),
  switchMap(code =>
    couponService.validate(code)
  )
);
```

Memory

Coupon/search input = `debounce + distinct + switch`

18. Multiple Events → Same Action

Suppose these all need cart refresh:

```
Add
Remove
Update Quantity
```

Use:

```
merge(
  addToCart$,
  removeFromCart$,
  updateQuantity$
```

```
).pipe(  
  switchMap(() =>  
    cartService.getCart()  
  )  
);
```

Memory

Any source can trigger = `merge`

19. Sequential Observables

```
concat(  
  saveProduct$,  
  logActivity$,  
  showSuccess$  
);
```

Memory

Observable A → B → C = `concat`

20. First Observable to Respond

```
race(  
  cache$,  
  api$  
);
```

Memory

First one wins = `race`

Use carefully; the fastest response isn't necessarily the most authoritative.

21. `map`

Transform values.

```
products$.pipe(  
  map(products =>  
    products.map(p => p.name)  
  )  
);
```

Memory

`map` = transform

22. `filter`

Allow only required values.

```
products$.pipe(  
  filter(product => product.inStock)  
);
```

Memory

`filter` = should this continue?

23. `tap`

Side effects.

```
products$.pipe(  
  tap(() => console.log('Products loaded'))  
);
```

Good for:

Logging
Loading **state**
Analytics
Debugging
Store dispatch
Navigation

Don't use **tap** to transform data.

24. **scan**

Maintain state over time.

```
productViewed$.pipe(  
  scan((products, product) => [  
    product,  
    ...products.filter(p => p.id !== product.id)  
  ].slice(0, 10), [])  
);
```

Good for:

Cart **accumulation**
Infinite **scroll**
Recent **products**
Counters
Running **totals**

Memory

| **scan** = continuously accumulating state

25. **debounceTime**

Wait for silence.

```
search$.pipe(  
  debounceTime(300)  
);
```

Good for

Search
Autocomplete
Coupon validation
Filter inputs

Memory

"Stop typing, then act."

26. throttleTime

Allow one value during a time window.

```
scroll$.pipe(  
  throttleTime(200)  
);
```

Good for:

Scroll
Mouse movement
Resize
High-frequency events

Difference

```
debounceTime → wait until quiet  
throttleTime → limit frequency
```

27. `distinctUntilChanged`

Prevent duplicate consecutive values.

```
category$.pipe(  
  distinctUntilChanged()  
);
```

Good for:

```
Search  
Category  
Sort  
Price  
Order status  
Inventory
```

28. `startWith`

Provide initial value.

```
category$.pipe(  
  startWith('all')  
);
```

Very useful with:

```
combineLatest(...)
```

Memory

`startWith` = initial/default value

29. `pairwise`

Previous + current.

```
price$.pipe(  
  pairwise()  
);
```

Produces:

```
[previous, current]
```

Good for:

```
Price changes  
Quantity changes  
Order status changes  
Filter changes
```

30. **take**

Take a fixed number.

```
config$.pipe(  
  take(1)  
);
```

Good for:

```
Load configuration once  
Get current user once  
Initial data
```

31. **takeUntil**

Stop when another observable emits.

```
products$.pipe(  
  takeUntil(destroy$)  
);
```

Good for:

WebSocket
Polling
Long-lived subscriptions
Component cleanup

Angular modern approach:

```
products$.pipe(  
  takeUntilDestroyed()  
);
```

32. `catchError`

Recover from errors.

```
productService.getProducts().pipe(  
  catchError(() => of([]))  
);
```

Important

`catchError` must return an observable.

```
catchError(() => of([]))
```

33. `retry`

Retry transient failures.

```
api$.pipe(  
  retry(2)  
);
```

Or:

```
api$.pipe(  
  retry({  
    count: 3,  
    delay: 1000  
  })  
);
```

Don't blindly retry

Especially avoid blindly retrying non-idempotent operations such as:

```
Payment  
Create Order  
Place Order
```

34. finalize

Runs when observable completes/errors/unsubscribes.

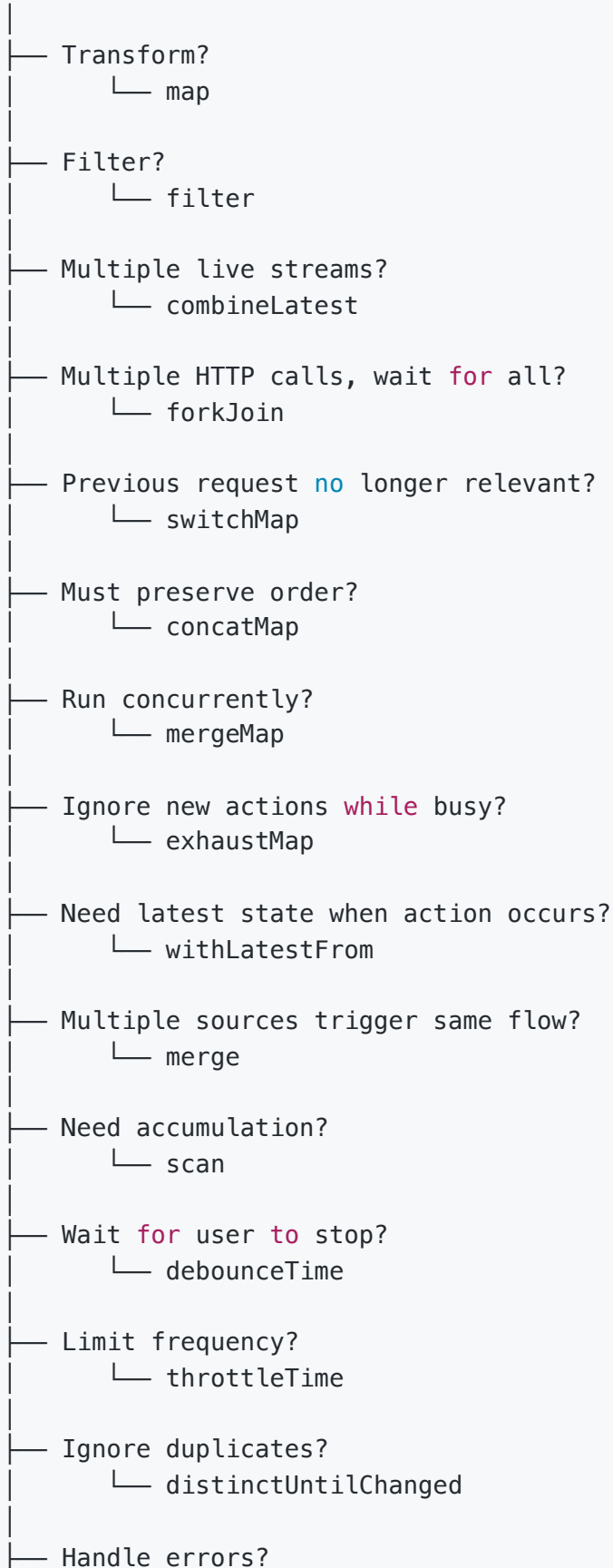
```
api$.pipe(  
  finalize(() => {  
    this.loading = false;  
  })  
);
```

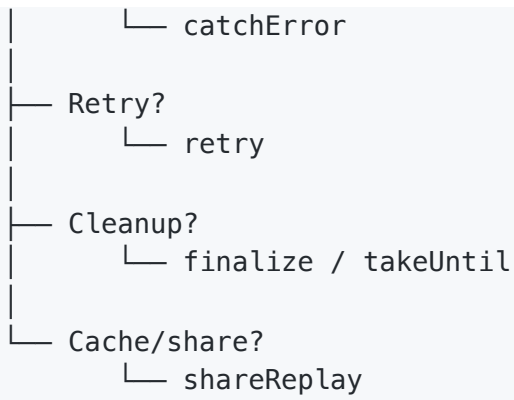
Perfect for

```
API started → spinner ON  
API ended   → spinner OFF
```

35. Operator Decision Tree

What are you trying **to do**?





36. E-commerce Master Map

SEARCH

debounceTime
distinctUntilChanged
filter
switchMap
catchError

FILTERS

combineLatest
debounceTime
distinctUntilChanged
switchMap

PRODUCT DETAILS

switchMap
forkJoin
shareReplay

CART

combineLatest
concatMap
scan
map

ADD TO CART

exhaustMap

PAYMENT

exhaustMap

CHECKOUT

concatMap
switchMap

- catchError
- finalize

INFINITE SCROLL

- filter
- exhaustMap
- scan

ORDER TRACKING

- timer
- switchMap
- distinctUntilChanged
- takeUntil

LIVE INVENTORY

- filter
- map
- distinctUntilChanged
- shareReplay

CACHE

- shareReplay

ACTION + STATE

- withLatestFrom

MULTIPLE EVENTS

- merge

INDEPENDENT HTTP

- forkJoin

LIVE SOURCES

- combineLatest

ERROR

- catchError

RETRY

- retry

LOADING

- tap
- finalize

37. 15 Operators to Master First

For a Senior Angular interview, prioritize these:

1. `switchMap`
2. `concatMap`
3. `mergeMap`
4. `exhaustMap`
5. `combineLatest`
6. `forkJoin`
7. `withLatestFrom`
8. `debounceTime`
9. `distinctUntilChanged`
10. `catchError`
11. `shareReplay`
12. `scan`
13. `tap`
14. `finalize`
15. `takeUntil` / `takeUntilDestroyed`

Final Memory Formula

SEARCH

= `debounce` + `distinct` + `switch`

FILTER

= `combineLatest` + `switch`

DEPENDENT API

= `switchMap`

INDEPENDENT APIs

= `forkJoin`

ORDERED REQUESTS

= `concatMap`

PARALLEL REQUESTS

= `mergeMap`

PREVENT DOUBLE CLICK

= `exhaustMap`

ACTION + CURRENT STATE

= `withLatestFrom`

ACCUMULATE

= `scan`

CACHE
= shareReplay

ERROR
= catchError

RETRY
= retry

CLEANUP
= takeUntilDestroyed

LOADING
= finalize

The Golden Interview Question

When an interviewer gives you an RxJS problem, ask yourself:

1. Are the operations independent **or** dependent?
2. Should previous work be cancelled?
3. Does order matter?
4. Can operations run **in** parallel?
5. Should new actions be ignored **while** busy?
6. Are the sources continuous **or** one-shot?
7. Do I need all results **or** latest results?
8. Can duplicate values be ignored?
9. Should input be debounced?
10. How will errors **and** cleanup be handled?

If you can answer those **10 questions**, choosing the correct RxJS operator becomes much easier.